

Parallel Programming with OpenMP - part1

Academic year 2025-2026

Prof. Massimo Torquati
massimo.torquati@unipi.it

Outline

- Introduction to OpenMP
- OpenMP directives for parallelizing loops (loop-parallelism)
- OpenMP primitives and environment variables
- Thread Affinity in OpenMP

OpenMP – Open Multi-Processing

1/2

- OpenMP is an API for **platform-independent** shared-memory parallel programming in C, C++, and Fortran, providing **high-level parallel abstractions** on top of low-level threading mechanisms
- OpenMP extends C, C++, and Fortran programming languages with directives (i.e., **#pragma omp** ...), a few library routines (e.g., **omp_get_thread_num()**), and environmental variables (e.g., **OMP_SCHEDULE**, **OMP_NUM_THREADS**)
- It is natively supported by almost all compilers (GCC, Intel, Clang,...). It is used to exploit shared-memory parallelism on CMPs (Chip Multi-Processors)
 - The compiler automatically generates the necessary multithreaded code and all needed synchronizations based on the directives specified in the source code

```
#pragma omp parallel for  
for(int i=0;i<N; ++i)  
    foo(i);
```

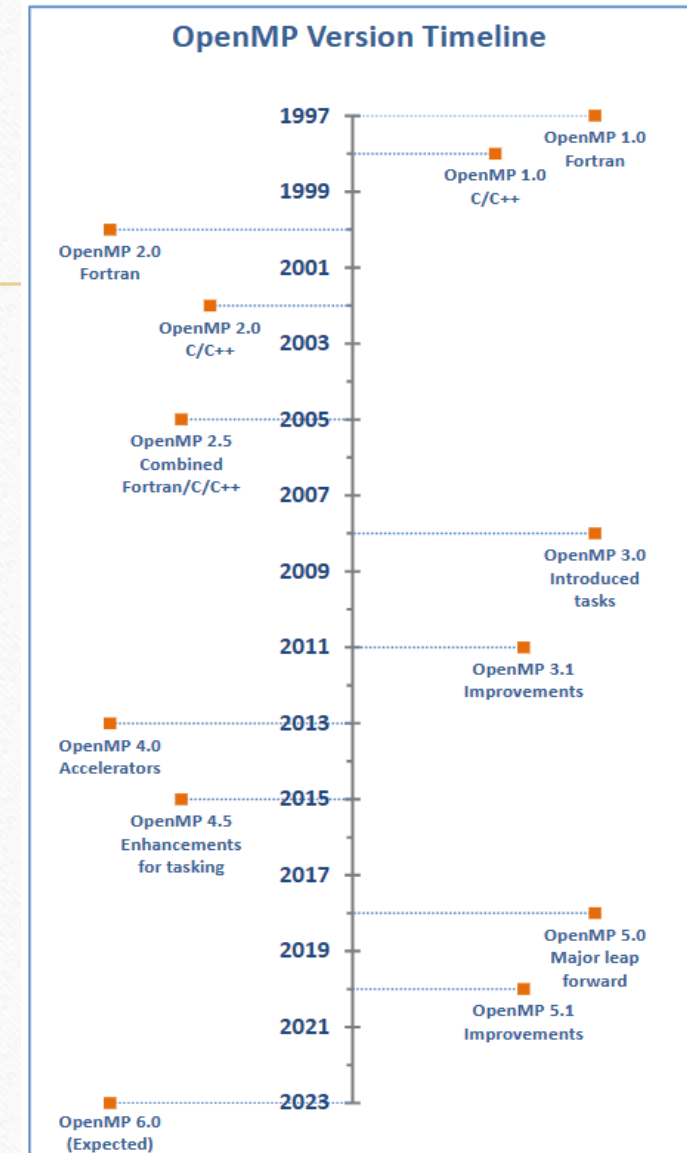

OpenMP – Open Multi-Processing

2/2

- OpenMP allows programmers to organize a program into serial and parallel regions. It provides synchronization constructs through a “**lightweight**” syntax (compiler directives)
- OpenMP **is NOT** a parallelizing compiler (i.e., it does not automatically parallelize sequential code!)
 - **Parallelism is explicit**
 - **Avoiding data races and obtaining good speedup is the programmer’s responsibility**

Brief History of OpenMP

- Started in the late 90s by a group of vendors, Intel, and the U.S. Department of Energy
- Version 1.0 released in '97. It supported only Fortran
- Binding for C/C++ was introduced later
- Initially the target was mainly loop parallelization
- In 2008 OpenMP 3.0 introduced **task parallelism**, which has been refined and improved in later versions
- In 2013 OpenMP 4.0 introduced the initial support for **accelerators** (i.e., GPGPUs)



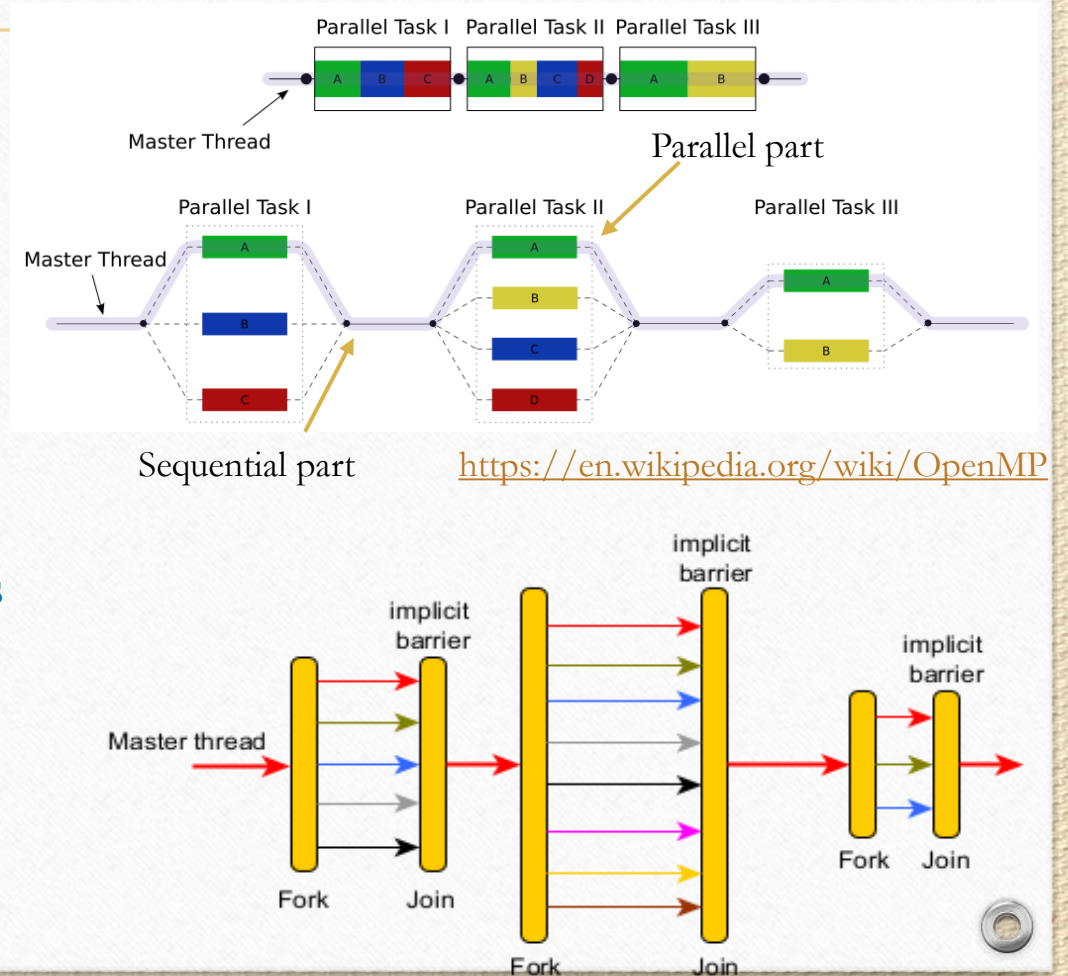
<https://cvw.cac.cornell.edu/openmp/intro/history-evolution>

OpenMP targets ease of use

- OpenMP does not require that single-threaded code be restructured for threading
 - It preserves sequential equivalence
 - It enables incremental parallelization of sequential programs
- OpenMP relies mainly on compiler directives
 - If the compiler does not recognize a directive, it ignores it
 - Parallelization is possible using just a small number of directives (both for coarse-grain and fine-grain parallelism)
- If the compiler is not instructed to process OpenMP directives (i.e., `-fopenmp`), the program will execute sequentially
 - Runtime routines have default sequential implementations

OpenMP execution model (Fork-join)

- Higher-level than C++/POSIX threads
 - Implicit mapping and load balancing of tasks
- The execution starts with a single thread called **Master thread** (now called the **Initial/Primary thread**)
- When a **parallel region** is encountered, that thread forms a **team** (or pool) of Worker threads, and the parallel region runs concurrently on the team
- Inside the region, work is created via **work-sharing constructs** and **tasks**; any team thread may execute tasks
- At the end of a parallel region there is an **implicit barrier synchronization**; after the barrier only the Initial thread continues the execution
- The **thread pool is reused** for next parallel regions



OpenMP Memory Model

- OpenMP provides a relaxed consistency memory model
 - It allows the reordering of accesses within a thread to different variables
 - A thread may not immediately see other threads' writes
- A thread may observe stale values unless synchronization is used
 - Registers, caches, compiler reordering, and hardware reordering all contribute to this effect
 - Additionally, each thread has its *threadprivate* memory that cannot be accessed by other threads
- **The Happens-before is established by synchronizations (barrier, critical, atomics, locks, task wait, etc.)**
- The **flush** directive is used to enforce consistency between a thread's temporary view of memory and the primary memory (or between multiple threads' views of memory)
 - `#pragma omp flush [variable list]`
 - Use explicit flushes with care since they can evict cached values. Subsequent accesses might reload data from memory.
- **Flush operations are implied by all synchronization operations in OpenMP**
 - **flush** is mainly a low-level visibility primitive and is rarely needed in well-structured OpenMP code
 - E.g., barrier, at entering/exiting to/from a parallel region, at the exit from workshare regions, etc. ...

OpenMP directives

- C and C++ compilers use **#pragma omp** prefix
 - Pragas are compiler directives enabling behaviors that are not part of the language
 - Directives are active only when OpenMP is enabled (i.e., -fopenmp)
 - Compilers that do not support **#pragma omp** ignore it
 - Fortran uses *significant comments* **!\$omp**, **c\$omp**, ***\$omp** prefixes
- Structure:

prefix directive [**clause list**]

 - Examples:
 - C/C++: **#pragma omp parallel** num_threads(8)
 - Fortran: **!\$omp parallel** num_threads(8)
 - Most OpenMP constructs apply to the **structured block** following the directive
 - A structured block is a block of statements with one entry point and one exit point (returning or jumping inside a parallel block is not allowed)
 - If the structured block is not explicitly marked (using braces {}), it is taken **the next single statement**

«Hello world»

```
#include <omp.h> // needed only for using library routines
```

```
int main() {  
    #pragma omp parallel  
    { // <- spawning of threads (Workers)  
        int i = omp_get_thread_num();  
        int n = omp_get_num_threads();  
        std::printf("Hello from thread %d of %d\n", i, n);  
    } // <- joining of threads  
}
```

```
[torquati@spmln spmcode6]$ g++ omp_hello.cpp -o omp_hello -fopenmp  
[torquati@spmln spmcode6]$  
[torquati@spmln spmcode6]$ OMP_NUM_THREADS=6 ./omp_hello  
Hello from thread 0 of 6  
Hello from thread 3 of 6  
Hello from thread 4 of 6  
Hello from thread 5 of 6  
Hello from thread 2 of 6  
Hello from thread 1 of 6
```

Library call, returns the ID of the Worker thread

Library call, returns the number of threads in the region

- The `-fopenmp` compiler flag enables OpenMP directives in GCC/Clang
- The environment variable `OMP_NUM_THREADS` can be used to set the default number of threads in the teams of threads that will be created for the parallel regions
 - In this example if it is not set, the number of threads in the team is equal to the number of available cores

«Hello world» (2)

- A structured block can be a single function call (in this example a C++ lambda)
- The `-fopenmp` compiler flag defines the preprocessor variable `_OPENMP`
 - Can be useful to include/exclude code at compile time
- The clause `num_threads` is used to set the number of threads to create for a given region

```
omp_hello2.cpp > ...
1  #include <stdio>
2  #include <string>
3  #if defined( OPENMP)
4  #include <omp.h>
5  #endif
6
7  int main(int argc, char *argv[]) {
8      auto F=[]() {
9          int i=0;
10         int n=1;
11         #if defined(_OPENMP)
12             i = omp_get_thread_num();
13             n = omp_get_num_threads();
14         #endif
15         std::printf("Hello from thread %d of %d\n", i, n);
16     };
17     int nth=(argc>1)?std::stoi(argv[1]):1;
18     #pragma omp parallel num_threads(nth)
19     F();
20 }
```

«Hello world» (3)

- The **if** clause is evaluated and if its expression evaluates to *true*, the parallel construct is enabled with **num_threads** thread, otherwise is ignored
- To measure the time, OpenMP provides a library routine **omp_get_wtime()**, which returns wall clock time in seconds

```
omp_hello3.cpp > ...
1  #include <stdio>
2  #include <string>
3  #include <thread>
4  #include <chrono>
5  #include <omp.h>
6
7  int main(int argc, char *argv[]) {
8      using namespace std::chrono_literals;
9      auto F=[](int i) {
10         std::this_thread::sleep_for(100ms);
11         std::printf("Hello from thread %d\n", i);
12     };
13     int nth=(argc>1)?std::stoi(argv[1]):1;
14     double start=omp_get_wtime();
15
16     #pragma omp parallel if (nth>=5) num_threads(nth)
17     F(omp_get_thread_num());
18
19     double stop=omp_get_wtime();
20     std::printf("Time %.2f (ms)\n", (stop-start)*1000.0);
21 }
```


OpenMP parallel directive

```
#pragma omp parallel [clause list]
```

- When a thread reaches a **parallel** directive, it creates a team of threads and becomes the primary thread of that team (the thread with ID 0)
- Each thread of the team computes the code of the parallel region (i.e., the structured block of the region)
- **At the end of the parallel region, there is an implicit barrier**
 - Additional synchronization constructs exist and can override or refine the implicit barrier
- Some commonly used clauses:
 - **if (scalar expression)**: determines whether the directive creates threads
 - **num_threads (integer expression)**: number of threads to create in the team
 - **private (variable list)**: specifies variables local to each thread, **uninitialized** on entry
 - **firstprivate (variable list)**: similar to **private**, the variables are **initialized** to variable value before the parallel directive
 - **shared (variable list)**: specifies that variables are shared among all threads in the team
 - In C/C++, most variables visible from an outer scope are shared by default
 - Loop iteration variables in for loops are private by default
 - **default (data scoping specifier)**: specifies the default data-sharing attribute for variables in the region. Common specifiers include **shared** and **none**
 - **reduction (operator : variable list)**: specifies how per-thread copies of in the listed variables are combined into a single copy at region exit

OpenMP worksharing constructs

- Inside a **parallel** region, a **worksharing construct** tells the threads in the team how to execute statements, blocks, and loop iterations cooperatively
- There are four **worksharing directives**:
 - **for**: threads cooperatively execute loop iterations
 - **sections**: threads cooperatively execute code blocks
 - **single**: one thread executes the block, other threads wait at the implicit barrier unless **nowait** is used
 - **workshare**: divides the execution of enclosed structured blocks into separate units of work (**Fortran only**)
- **for** directive:
 - `#pragma omp for [clause list]`
 - It partitions loop iterations across team threads
 - Common clauses in [clause list]
 - **private**, **firstprivate**, **lastprivate**
 - **reduction**
 - **schedule**, **collapse**, **nowait**, and **ordered**
 - At the end of the loop, there is an implicit barrier
 - It requires the loop to be in **canonical form**
 - <https://www.openmp.org/spec-html/5.0/openmpsu40.html>

Example using **parallel** and **for**

```
#pragma omp parallel
{ // <- spawning of threads
  #pragma omp for
  for (uint64_t i = 0; i < num_entries; i++) {
    x[i] = i;
    y[i] = num_entries - i;
  }
  // <- implicit barrier
  #pragma omp for
  for (uint64_t i = 0; i < num_entries; i++)
    z[i] = x[i] + y[i];
  // <- another implicit barrier
  #pragma omp for
  for (uint64_t i = 0; i < num_entries; i++)
    if (z[i] - num_entries)
      std::cout << "error at position " << i << std::endl;
} // <- joining of threads (final barrier)
```

Vector addition spawning threads once at the beginning of the parallel region and joining at exit

Avoiding implicit barriers when not needed (**nowait** clause), i.e., when loops are independent

```
#pragma omp parallel
{ // <- spawning of threads
  #pragma omp for nowait
  for (...) { ... }

  // <- here there is no barrier

  #pragma omp for
  for (...) { ... }
} // <- joining of threads (final barrier, cannot be removed)
```

Any thread can begin the second loop immediately without waiting for other threads to finish the first loop

Variable sharing and privatization

- Variables scope:

- shared(x)**: all threads access the same memory location of x
 - Many variables from an enclosing scope are shared by default, but not all
- private(x)**: each thread has its private copy of x
 - Private copies are **not** initialized
 - Local updates to x are discarded when exiting the parallel region. **At the end of the region, x retains its original value**
- firstprivate (x)**: each thread has its own private copy of x
 - All local copies of x are **initialized** with the value that x has before starting the parallel region
 - Local updates to x are discarded when exiting the parallel region. **At the end of the region, x retains its original value**
- lastprivate (x)**: primarily used with **for** and **sections**. It has the same semantics as **private(x)**, but at the end of the construct, x retains the value from the logically last iteration or **section**
- default(shared)** or **default(none)**: set the default data sharing of variables not specified in other clauses
 - default(none)**: ensures that you must specify the scope of each variable in the parallel block. Prefer **default(none)**

```
int x=10, last = -1;
#pragma omp parallel for firstprivate(x) lastprivate(last)
for (int i=0; i<4; ++i) {
    x += i; // modifies thread's private x
    last = i; // private, but copied out at the end
}
// here x=10, last = 3
```


Example 1

```
scope.cpp > ...
1  #include <stdio>
2  #include <omp.h>
3
4  int main() {
5
6      int a=0;
7      int b=1;
8      int c=2;
9      int d=3;
10
11  #pragma omp parallel num_threads(8) private(a) shared(b) firstprivate(c)
12  {
13      a++; // not initialized! Warning from compiler (-Wall)
14      b++; // this is not atomic
15      c++; // local copies initialized
16      d++; // this is shared by default, non atomic increment
17      std::printf("Hi %d a=%d, b=%d, c=%d, d=%d\n",
18                  omp_get_thread_num(), a,b,c,d);
19  }
20  std::printf("Final values: a=%d b=%d c=%d d=%d\n", a, b, c, d);
21
22 }
```

```
[torquati@spmln spmcode6]$ ./omp_scope
Hi 0 a=1, b=3, c=3, d=5
Hi 4 a=1, b=2, c=3, d=4
Hi 2 a=1, b=3, c=3, d=5
Hi 5 a=1, b=2, c=3, d=4
Hi 6 a=1, b=2, c=3, d=4
Hi 7 a=1, b=4, c=3, d=6
Hi 3 a=1, b=2, c=3, d=4
Hi 1 a=1, b=2, c=3, d=4
Final values: a=0 b=4 c=2 d=6
```

b and d are shared, but their increments are **unsynchronized**, so the final values are not deterministic. a is **private** and uninitialized inside the region, while c is **firstprivate** and therefore initialized from its original value

Example 2

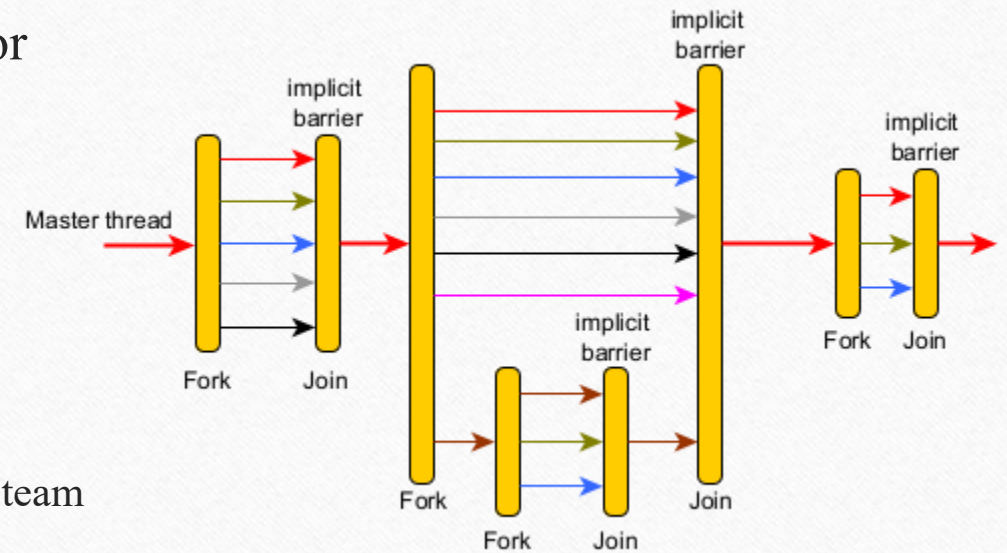
```
scope2.cpp > ...
1  #include <stdio>
2  #include <omp.h>
3
4  int main() {
5
6      int i;
7
8      #pragma omp parallel for lastprivate(i) num_threads(8)
9          for(int j=0;j<16; ++j) {
10          i=j;
11          std::printf("Hi %d i=%d\n",omp_get_thread_num(), i);
12      }
13      std::printf("Final value: i=%d\n", i);
14
15 }
```

```
[torquati@spmln spmcode6]$ ./omp_scope2
Hi 5 i=10
Hi 5 i=11
Hi 0 i=0
Hi 0 i=1
Hi 3 i=6
Hi 3 i=7
Hi 2 i=4
Hi 2 i=5
Hi 1 i=2
Hi 1 i=3
Hi 4 i=8
Hi 4 i=9
Hi 7 i=14
Hi 7 i=15
Hi 6 i=12
Hi 6 i=13
Final value: i=15
```

`i` is made **lastprivate** on the parallel for, so each iteration has a private copy of `i`, and the value from the logically last iteration (here `j==15`) is copied back at the end. **Final print should show `i == 15`, regardless of scheduling.**

Nested parallelism

- Nested parallelism enabled by using the `OMP_NESTED` or `omp_set_nested(1)`
 - **Nested parallelism is often disabled by default**
 - E.g., `OMP_NESTED=true ./myprog` or `omp_set_nested(1)`
 - `omp_get_num_threads()` returns the size of the current team
 - NOTE: if no parallel region is active, it returns 1
 - `omp_get_thread_num()` returns the calling thread ID in the current team
 - `omp_get_max_threads()` returns the maximum number of threads that can be created
- **Nested parallelism is rarely used in practice**
 - Beware to oversubscription, control it with `OMP_THREAD_LIMIT`



`OMP_MAX_ACTIVE_LEVELS=2` - allow 2 active levels
`OMP_NUM_THREADS=4,2` - request 4 outer and 2 inner threads
`OMP_THREAD_LIMIT=8` - cap total threads across levels

Example: `omp_nested.cpp`

reduction clause in the parallel directive

- It specifies the **associative operator** to use to combine local copies of a variable in different threads into a single value at the end of the construct
- Reduction operators: +, *, |, ^, &&, ||, *min*, *max*
 - **Identity values**: 0, 1, 0, 0, 1, 0, INT_MAX, INT_MIN (for int)
- One private copy of each listed variable in the **variable list** is created for each thread.
- Each copy is initialized with the **neutral element of the reduction operator** (e.g., 0 for +), then each thread executes the parallel region.
- At the end of the region, the private copies are combined using the reduction operator, together with the original value of the reduction variable before entering the region

```
#pragma omp parallel reduction (op : variable list)
```

```
int x=5;
int y=1;
#pragma omp parallel reduction(* : x) \
                        reduction(+ : y) num_threads(3)
{ // <- spawning 3 threads, private x,y set to 1 and 0
  x += 3;
  y += 3;
} // <- joining threads
// the value of x here is 320 ( (1 + 3)3 × 5 )
// the value of y here is 10 (3 × 3 + 1 )
```

Example: omp_reduction.cpp omp_dotprod.cpp

Example: estimation of π

- Mathematically, we know that:

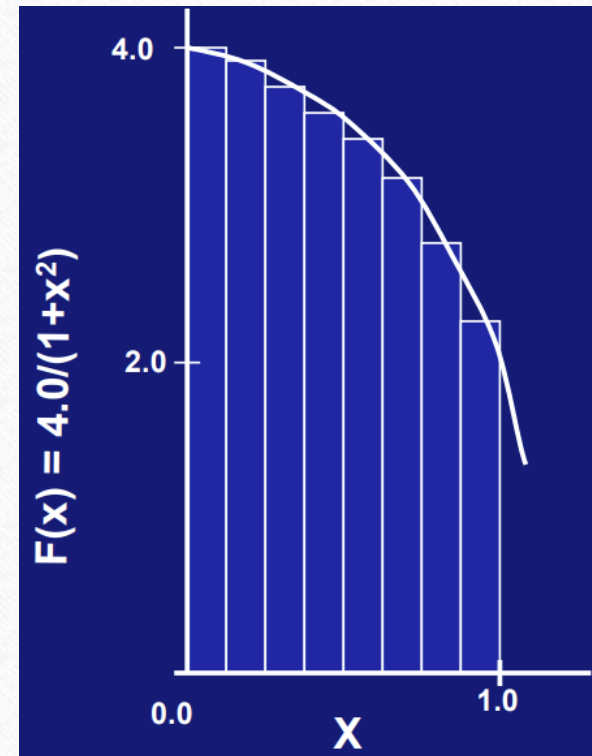
$$\int_0^1 \frac{4.0}{(1+x^2)} dx = \pi$$

- It is possible to approximate the integral as a sum of the N rectangles in $[0,1]$ where each rectangle has width Δx and height $F(x_i)$, with $F(x) = \frac{4.0}{(1+x^2)}$, computed at the midpoint of interval i :

$$\pi \approx \sum_{i=0}^N F(x_i) \times \Delta x$$

- Another option is to compute N steps of the following sum:

$$\pi = 4 \sum_{k=0}^{+\infty} \frac{(-1)^k}{(2k+1)}$$



Example: estimation of π

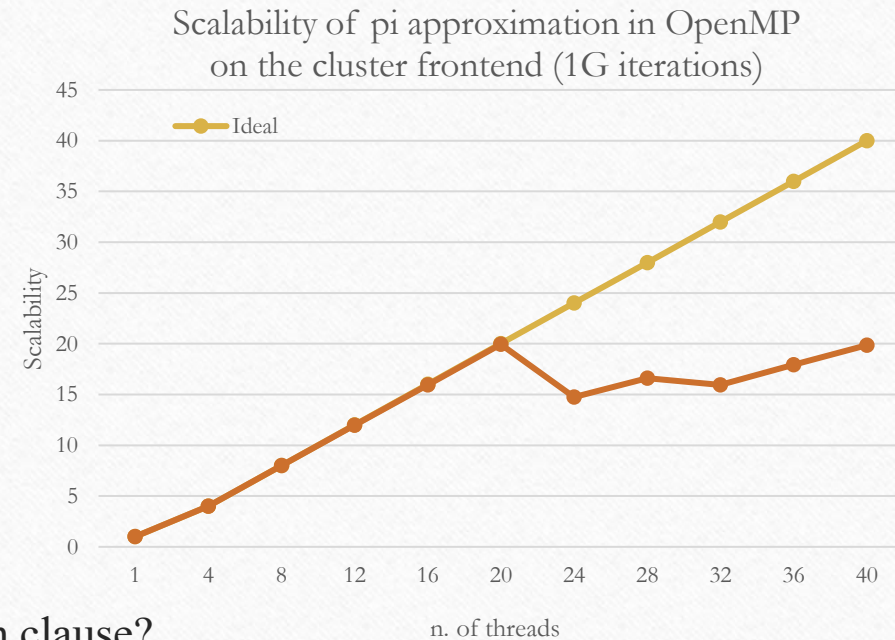
```
uint64_t num_steps = std::stol(argv[1]);
long double x = 0.0;
long double pi = 0.0;
long double sum = 0.0;
long double step = 1.0/num_steps;

double start = omp_get_wtime();

#pragma omp parallel for private(x) reduction(+:sum)
for (uint64_t k=0; k < num_steps; ++k) {
    x = (k + 0.5) * step;
    sum += 4.0/(1.0 + x*x);
}
pi = step * sum;

double elapsed = omp_get_wtime() - start;
```

- x must be private
- sum (i.e., the reduction variable) is automatically privatized



How to implement the estimation of π without using the reduction clause?
(take a look at *omp_pi-noreduction.cpp* file)

Mapping iterations to threads

```
#pragma omp parallel for schedule(scheduling_class[,chunksize])
```

- The **schedule** clause enables different loop iterations distribution policies (iterations scheduling). Four classes:
 - **static**: work assigned statically to threads before loop execution in a cyclic fashion.
 - If **chunksize** is not set, then each thread receives about $\left\lceil \frac{\#iterations}{nthreads} \right\rceil$ contiguous iterations; otherwise, the iteration space is divided into blocks of size **chunksize** assigned round-robin
 - **dynamic**: iterations assigned dynamically at run time at a granularity of **chunksize** (default **chunksize**=1)
 - When a thread completes one chunk, another one (if available) is assigned to it. Good for unbalanced workloads, but higher overhead.
 - **guided**: iterations assigned dynamically at run time at a granularity that is exponentially reduced with each dispatched piece of work up to chunksize (the default minimum chunksize is 1)
 - The actual implementation depends on the compiler and runtime. Usually, less overhead than **dynamic**, but even less deterministic
 - **runtime**: the scheduling policy depends on the OMP_SCHEDULE environment variable
 - E.g., OMP_SCHEDULE="dynamic,10" ./myprog
 - **auto**: delegates the decision of the scheduling to the compiler and the runtime system

The default schedule is implementation dependent. In GCC it is **static**.

Static schedule examples

```
#pragma omp parallel for [schedule policy] num_threads(5)
for (uint64_t i = 0; i < 16; i++) {
    ...
}
```



- `schedule(static)`



- `schedule(static, 1)`

cyclic distribution



- `schedule(static, 2)`

block-cyclic distribution with $c=2$



Dynamic schedule

- For **dynamic**, iterations are assigned in chunks of **chunksize** consecutive iterations. For **guided**, chunk sizes decrease over time down to a minimum **chunksize**
 - In the **guided** schedule, the **chunksize** is a lower bound (**chunkmin**). Bigger chunks are assigned at the beginning, and as chunks are completed, the size of the new chunks decreases down to **chunkmin**
 - Guided heuristic: $\max\left(\left\lceil \frac{\#remainingiteration}{\#threads} \right\rceil, chunkmin\right)$
 - When a thread finishes executing a chunk, a new chunk is assigned by the runtime system
- `schedule(dynamic,1)`

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
---	---	---	---	---	---	---	---	---	---	----	----	----	----	----	----

 (not predictable)
- Conceptually, it is a master-worker computation paradigm where the master dispatches tasks to ready Workers
 - The dynamic schedule requires more work at runtime (thus more overhead than the static schedule), but usually it provides better workload balancing among Workers if there are unpredictable or highly variable work per iteration
 - The chunksize is another critical factor, the optimal value is system- and application-dependent

Guided example

```
#pragma omp parallel for schedule(guided,8) num_threads(4)
for (uint64_t i = 0; i < 100; i++) {
    ...
}
```

- One possible schedule, actual assignment is implementation-dependent:
 - Th0 : $\max\left(\left\lceil\frac{100}{4}\right\rceil, 8\right) = 25$, remaining 75, assigned iterations: 0-24
 - Th1: $\max\left(\left\lceil\frac{75}{4}\right\rceil, 8\right) = 19$, remaining 56, assigned iterations: 25-43
 - Th2: $\max\left(\left\lceil\frac{56}{4}\right\rceil, 8\right) = 14$, remaining 42, assigned iterations: 44-57
 - Th3: $\max\left(\left\lceil\frac{42}{4}\right\rceil, 8\right) = 11$, remaining 31, assigned iterations: 58-68
 - Th3: $\max\left(\left\lceil\frac{31}{4}\right\rceil, 8\right) = 8$, remaining 23, assigned iterations: 69-76
 - Th2: $\max\left(\left\lceil\frac{23}{4}\right\rceil, 8\right) = 8$, remaining 15, assigned iterations: 77-84
 - Th3: $\max\left(\left\lceil\frac{15}{4}\right\rceil, 8\right) = 8$, remaining 7, assigned iterations: 85-92
 - Th1: 7, assigned iterations: 93-99
- $\text{chunk} = \max\left(\left\lceil\frac{\#remainingiteration}{\#threads}\right\rceil, \text{chunkmin}\right)$
- Large chunks at the beginning, and progressively smaller chunks down to the minimum chunksize
- Smaller overhead than dynamic, but also potentially worse load-balancing
- Locality is typically worse than with static scheduling

Example: All-pairs distance Matrix

- Results obtained running the *all_pair.cpp* implementation for the MNIST dataset on the cluster front-end node
- Sequential time: 704s
- `OMP_NUM_THREADS=40`

```
[torquati@spmln spmcode5]$ OMP_NUM_THREADS=40 OMP_SCHEDULE="static,8" ./all_pair 1  
running in parallel  
# elapsed time (load_data_from_disk): 0.0792011s  
# elapsed time (compute_distances): 44.1978s
```

Iteration scheduling (gcc 12.2.0)	
OMP_SCHEDULE	Time (s)
static	102.55
guided,1	56.89
static,8	44.20
static,1	38.90
dynamic,8	35.26
dynamic,1	33.88

Example: Mandelbrot Set

- Each pixel of the Mandelbrot Set can be computed independently
- To use a coarser granularity, it is possible to parallelize the outer loop (computing all pixels in a row)
- By using the clause `schedule(runtime)` we can experiment with different iteration scheduling policies by setting the `OMP_SCHEDULE` env variable
- `#pragma omp critical` marks a region to be a **critical section**. Threads in the team access the region in mutual exclusion. Here it is used to serialize the drawing of each row.

```
93  int main(int argc, char *argv[]) {
94      gfx_open(XSIZE, YSIZE, "Mandelbrot Set");
95
96      #pragma omp parallel for schedule(runtime) default(none) \
97          shared(XSIZE, YSIZE, XMAX, XMIN, YMAX, YMIN)
98          for (int y = 0; y < YSIZE; ++y) {
99          const double im = YMAX - (YMAX - YMIN) * y / (YSIZE - 1);
100          // temporary buffer to store the pixel values for the current line
101          std::vector<int> results(XSIZE);
102          for (int x = 0; x < XSIZE; ++x) {
103              const double re = XMIN + (XMAX - XMIN) * x / (XSIZE - 1);
104              results[x] = iterate(std::complex<double>(re, im));
105          }
106          #pragma omp critical
107              drawrowpixels(y, results);
108          #endif
109      }
110
111      TIMERSTOP(mandel_seq);
112      std::cout << "Click to finish\n"; gfx_wait();
113  }
```


The collapse clause

- It specifies how many loops in a loop nest are collapsed into one large iteration space and then divided according to the **schedule** clause
- The main objective is to increase the iteration count
- All collapsed loops should be in canonical form

```
#pragma omp parallel for schedule(static) \  
    num_threads(5) collapse(2)  
for (int i = 0; i < 4; ++i)  
    for(int j=0; j < 5; ++j)  
        do_something(i,j);
```

Iteration space

0,0	0,1	0,2	0,3	0,4
1,0	1,1	1,2	1,3	1,4
2,0	2,1	2,2	2,3	2,4
3,0	3,1	3,2	3,3	3,4

Th0

Th1

Th2

Th3

Th4

0,0	0,1	0,2	0,3	0,4
1,0	1,1	1,2	1,3	1,4
2,0	2,1	2,2	2,3	2,4
3,0	3,1	3,2	3,3	3,4

Collapsed iteration space with static iteration schedule

0,0	0,1	0,2	0,3	0,4	1,0	1,1	1,2	1,3	1,4	2,0	2,1	2,2	2,3	2,4	3,0	3,1	3,2	3,3	3,4
-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----

Useful when the outer loop has too few iterations to keep all threads busy

Example: omp_collapse.cpp

Worksharing **sections** directive

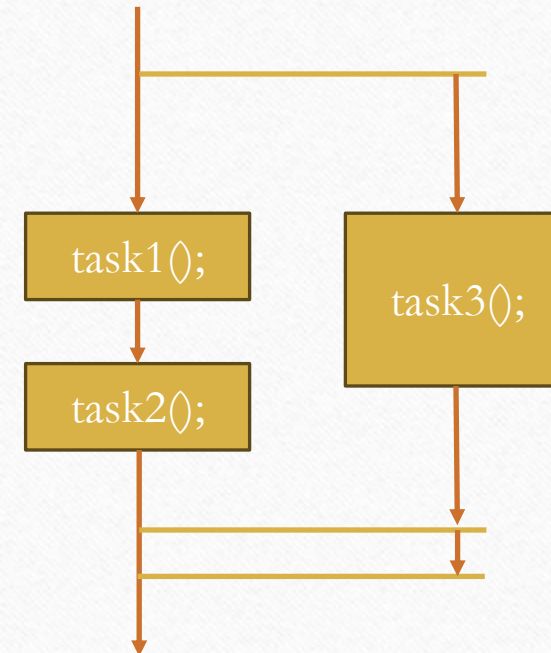
```
#pragma omp sections [clause list] {  
    [#pragma omp section  
    // structured block  
    ] // square brackets means optional!  
}
```

- The **sections** directive expresses a simple form of **task parallelism**. It is a work-sharing construct
 - Later we will discuss the **task** construct, **which is more flexible for irregular task parallelism**
- It specifies that the enclosed **section(s)** of the structured block are divided among team threads
 - Each **section** is executed once by one thread. Different **sections** may be executed by different threads
 - There is an **implicit barrier** at the end of a **sections** directive, unless the **nowait** clause is used
 - If used outside a parallel region, the code runs serially on the primary thread.
 - No schedule clause. Typical clauses are: **private**, **firstprivate**, **reduction**, **nowait**

Using the **sections** directive

```
#pragma omp parallel num_threads(2)
{
    #pragma omp sections
    {
        #pragma omp section
        {
            task1();
        }
        #pragma omp section
        {
            task2();
        }
        #pragma omp section
        {
            task3();
        }
    } // <- implicit barrier
} // <- implicit barrier
```

- The execution order of sections is not known
- The assignment of sections to threads is non-deterministic



Example

- Note the **nowait** clause on **sections**
This removes the implicit barrier at the end of the construct

```
omp_sections.cpp > ...
1  #include <stdio>
2  #include <omp.h>
3
4  int main(void) {
5      const int N=128;
6      float  a[N], b[N], c[N], d[N];
7
8      #pragma omp parallel for num_threads(4)
9          for (int i=0; i < N; i++) {
10         a[i] = i * 1.0;
11         b[i] = i * 3.14;
12     }
13     #pragma omp parallel num_threads(4)
14     {
15         #pragma omp sections nowait
16         {
17             #pragma omp section
18             {
19                 std::printf("Thread %d executing sec1\n", omp_get_thread_num());
20                 for (int i=0; i < N; ++i)
21                     c[i] = a[i] + b[i];
22             }
23             #pragma omp section
24             {
25                 std::printf("Thread %d executing sec2\n", omp_get_thread_num());
26                 for (int i=0; i < N; ++i)
27                     d[i] = a[i] * b[i];
28             }
29         } // <- no barrier here
30         } // <- implicit barrier here
31
32         float r=0.0;
33         #pragma omp parallel for reduction(+ : r)
34             for (int i=0; i < N; ++i)
35                 r += d[i] - c[i];
36
37         std::printf("%f\n", r);
38     }
```


Example: pipeline

- Pipeline example
- **Guaranteed**: each **section** is executed once by one thread.
- **Not guaranteed**: a one-to-one mapping between the 4 sections and the 4 threads
 - Disabling dynamic team-size can help to keep a fixed team, but it still does not enforce a 1-to-1 mapping (`OMP_DYNAMIC=false` or `omp_set_dynamic(0)`)
- If you need a strict 1-stage 1-thread mapping, see `omp_pipeline_nosections.cpp`, which uses a **parallel** region and an explicit check on the thread id
 - However, for general pipeline parallelism in OpenMP, **task** is usually the more appropriate construct

Example: `omp_pipeline_sections.cpp`

```
47 int main() {
48     const int N = 100;
49     BlockingQueue<Msg> q01(64), q12(64), q23(64); // queues between stages
50     long long total = 0;
51
52     omp_set_dynamic(0); // disable dynamic team-size adjustment
53
54     // Pipeline of 4 stages
55     // NOTE: one-to-one mapping of stages to threads not guaranteed
56     #pragma omp parallel sections num_threads(4) default(none) \
57         shared(N, q01, q12, q23, total, std::cout)
58     {
59         #pragma omp section // Stage 0: source, produce 1..N
60         {
61             for (int x = 1; x <= N; ++x) q01.push(Msg::data(x));
62             q01.push(Msg::stop());
63         }
64
65         #pragma omp section // Stage 1: simple map x -> x*x
66         {
67             for (;;) {
68                 Msg m;
69                 q01.pop(m);
70                 if (m.done) { q12.push(Msg::stop()); break; }
71                 q12.push(Msg::data(m.val * m.val));
72             }
73         }
74
75         #pragma omp section // Stage 2: filter, keep multiples of 3
76         {
77             for (;;) {
78                 Msg m; q12.pop(m);
79                 if (m.done) { q23.push(Msg::stop()); break; }
80                 if (m.val % 3 == 0) q23.push(Msg::data(m.val));
81             }
82         }
83
84         #pragma omp section // Stage 3: sink, compute the sum
85         {
86             for (;;) {
87                 Msg m; q23.pop(m);
88                 if (m.done) break;
89                 total += m.val;
90             }
91             std::cout << "Sum: " << total << "\n";
92         }
93     }
94 }
```

OpenMP synchronization constructs

`#pragma omp barrier` All threads in the active team must reach this point before they can proceed

`#pragma omp single [clause list]`
structured block

Mark a block to be executed **only by one thread** (the first one reaching it).
NOTE: like the other worksharing directives, there is an **implicit barrier** at the end of the block.

`#pragma omp master`
structured block

Marks a structured block to be executed **only by the Master thread** (the one with ID 0).
There is **no implicit barrier** at the end of the block.

`#pragma omp critical`
structured block

Mark the block to be a **critical section**. All threads will execute the critical section one at a time (lock-based). Use **critical(name)** for distinct critical sections.

`#pragma omp ordered`
structured block

It enforces sequential execution of the structured block, contained in a parallel loop, to be executed in loop order. Use it only if a small part of each iteration must respect the original loop order.

`#pragma omp atomic`

Performs a specific memory update atomically on a shared variable

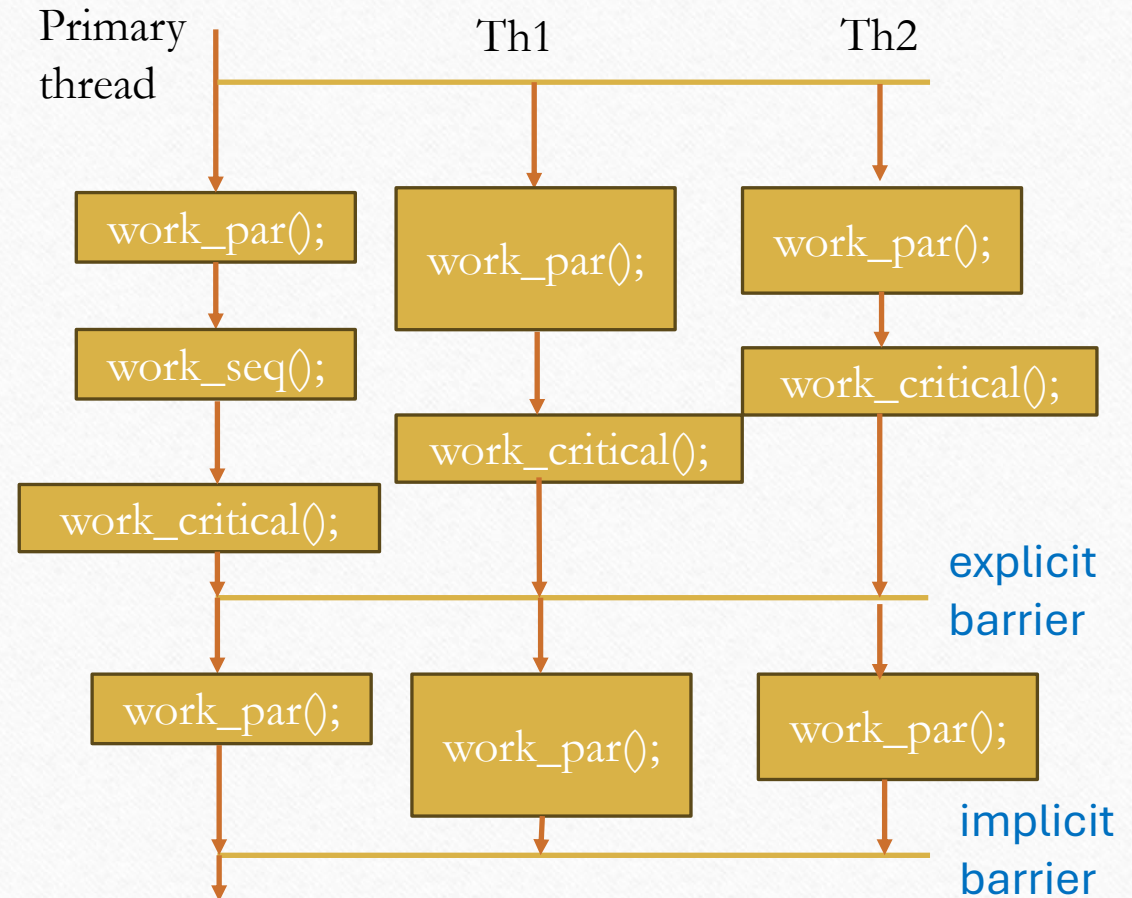
Using synchronization constructs

```
#pragma omp parallel
{
  work_par(); // all threads
  // run once on the master thread
  #pragma omp master
  work_seq();

  // mutual exclusion
  #pragma omp critical
  work_critical();

  #pragma omp barrier

  work_par(); // all threads
} // <- implicit barrier
```



Example: scan in OpenMP

- The input vector is partitioned into contiguous blocks, one block per thread
- Each thread computes a local scan on its own block
- `block_sum[tid]` stores the sum of block `tid`
- One thread computes the exclusive scan of `block_sum`
- Each thread adds the offset from the previous blocks to all elements of its local scan
- **Inclusive** scan algorithm , 4 threads, `op=+`

- x: -2 4 2 -1 -1 1 -3 2 1 4 1 5 initial vector
- y: -2 2 4 -1 -2 -1 -3 -1 0 4 5 10 local scan
 block_sum: 4, -1, 0, 10 block_sum[tid]
- block_sum: 0, 4, 3, 3 excl. scan (op=+)
- y: -2 2 4 -1 -2 -1 -3 -1 0 4 5 10
 + block_sum[tid] 0 4 3 3
 y: -2 2 4 3 2 3 0 2 3 7 8 13 final result

Scan is not a simple parallel for, but a multi-phase parallel pattern with local scans, coordination, and a final offset correction. Here we show how to implement it manually- [OpenMP 5.x](#) provides built-in support for **inclusive** and **exclusive** scan operations

Thread Affinity in OpenMP

- Thread Affinity concepts (recap):

- Thread affinity controls how threads are mapped to HW resources (CPU cores and sockets)
- Keeping a thread on a specific core helps maintain cache locality and reduce memory latency, particularly on NUMA architectures, and minimizes context switch overheads
- Affinity is particularly beneficial for memory-bound workloads
 - Pay attention to the **first touch** allocation policy

First touch with threading: array elements are allocated in the memory of the NUMA node containing the core that executes the thread which initializes the partition

- OpenMP Affinity management:

- **OMP_PROC_BIND** controls whether threads are allowed to migrate between cores. Values can be:
 - **true** or **false**: with false, the runtime may migrate threads among places; threads are bound to places, using the default policy
 - **close**: child threads are kept close to parent threads (good for cache locality)
 - **spread**: threads are spread among places to maximize distance (good for bandwidth)
 - **master**: child threads are placed as close as possible to the master thread
- **OMP_PLACES** defines a portable list of «places» (e.g., **threads**, **cores**, **sockets**) where threads can execute

Thread Affinity Examples

- `OMP_PLACES="cores"` defines one place per physical core
- `OMP_PLACES="threads"` defines one place per HW context
- `OMP_PLACES="sockets"` defines one place per socket
- `OMP_PROC_BIND="close"` `OMP_PLACES="cores"` packs threads on nearby cores, first filling one socket then the next one; improves cache locality
- `OMP_PROC_BIND="spread"` `OMP_PLACES="cores"` distributes threads across cores and sockets to maximize distance; can improve NUMA bandwidth and reduce cache contention
- Use `OMP_DISPLAY_ENV="true"` to see the settings of OpenMP env variables
- It is also possible to set binding per parallel region, e.g., `#pragma omp parallel proc_bind(close)`

```
OMP_PROC_BIND=close OMP_PLACES="cores" ./omp_affinity
```

```
OMP_PLACES="{0:8:1}, {8:8:1}" ./omp_affinity
```

Example: `omp_affinity.cpp`